

TCP Congestion Control

(Slow Start, Congestion Avoidance, and Fast Recovery)

1. Mechanisms of Sender Throttling

TCP uses **End-to-End Congestion Control**, meaning the sender must infer network conditions based on observed behavior (loss and delay) because the IP layer provides no explicit assistance.

The Congestion Window (cwnd)

The sender maintains a variable **cwnd** that limits the amount of unacknowledged data in flight.

The Sender Constraint

$$\text{LastByteSent} - \text{LastByteAcked} \leq \min\{\text{cwnd}, \text{rwnd}\}$$

*Assuming **rwnd** is large, **cwnd** becomes the primary bottleneck.*

Estimating Throughput

The transmission rate is roughly equal to **one window size per RTT**:

$$\text{Rate} \approx \frac{\text{cwnd}}{\text{RTT}} \text{ bytes/sec}$$

2. Congestion Detection: Implicit Signals

TCP uses ACKs to "clock" its growth (self-clocking) and loss events to trigger reductions.

- **Loss Event Type 1: Timeout** – Indicates severe congestion (potentially a silent network).
- **Loss Event Type 2: 3 Duplicate ACKs** – Indicates a gap (one segment lost, but others are getting through).

Guiding Principles

- **Lost segment** $\Rightarrow$ Decrease rate.
- **Acknowledged segment** $\Rightarrow$ Increase rate.
- **Bandwidth Probing** $\Rightarrow$ Increase until loss, then back off and repeat (sawtooth behavior).

3. The Core Algorithm (RFC 5681)

The algorithm consists of three mandatory phases (Slow Start, Congestion Avoidance) and one recommended phase (Fast Recovery).

3.1 Phase 1: Slow Start

Despite the name, growth is **exponential**.

- **cwnd** starts at **1 MSS**.
- **cwnd** increases by **1 MSS** for every **ACK** received.
- Effectively, **cwnd** **doubles** every **RTT**.

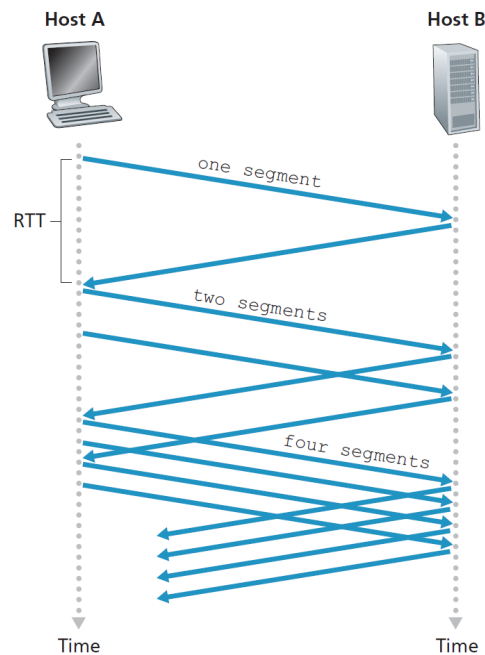


Figure 1: TCP Slow Start: Exponential Window Growth

Transitioning out of Slow Start

1. **Timeout:** Set **cwnd** = 1, **ssthresh** = **cwnd**/2, restart Slow Start.
2. **Meet Threshold:** When **cwnd** $\geq$ **ssthresh**, transition to **Congestion Avoidance**.
3. **3 Duplicate ACKs:** Perform fast retransmit and enter **Fast Recovery**.

3.2 Phase 2: Congestion Avoidance

On entry, **cwnd** is approximately half its previous pre-loss value. Growth is **linear**.

- **cwnd** increases by **1 MSS** every **RTT**.
- Formula: **cwnd** = **cwnd** + **MSS** · (**MSS**/**cwnd**) on each ACK.

3.3 Phase 3: Fast Recovery (Recommended)

- **cwnd** is increased by **1 MSS** for every **duplicate ACK** received for the missing segment.
- When a "new" ACK (acknowledging the missing data) arrives, TCP deflates the window and returns to Congestion Avoidance.

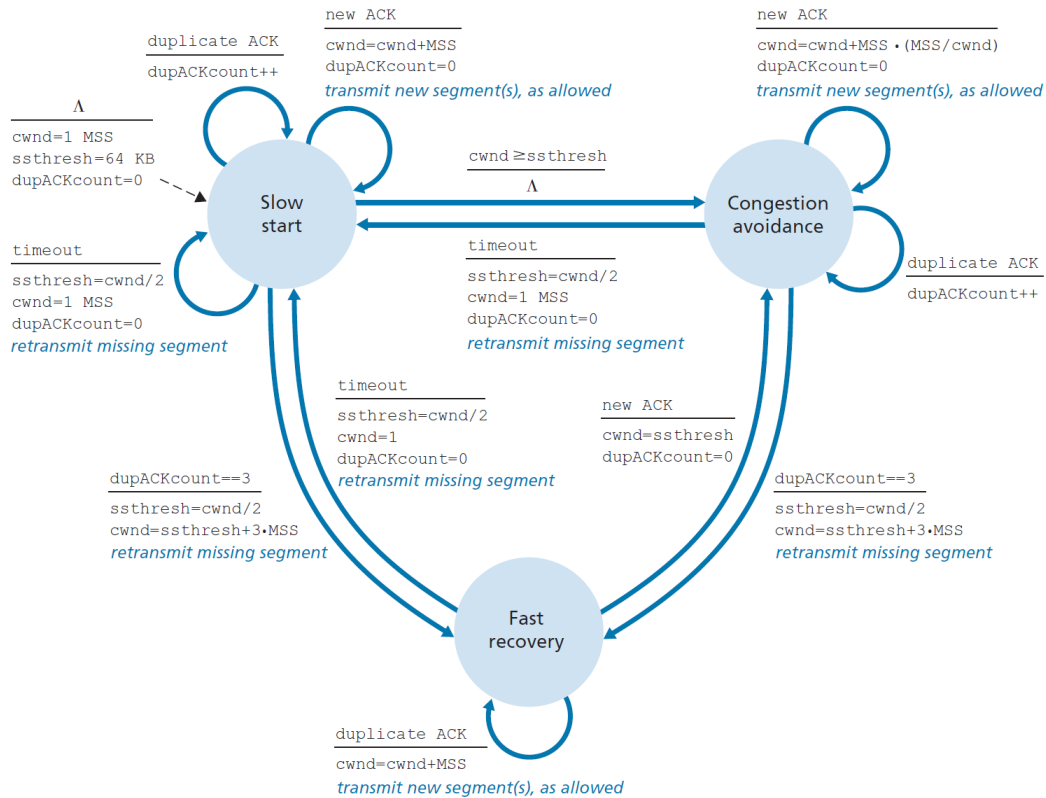


Figure 2: Complete TCP Congestion Control FSM

4. Evolution: TCP Tahoe vs. Reno

- **TCP Tahoe (Old)**: Always cuts $cwnd$ to 1 MSS on any loss (Timeout or 3-Dup-ACK).
- **TCP Reno (Modern)**: Only cuts $cwnd$ to 1 on **Timeouts**. On **3-Dup-ACKs**, it halves the window (Multiplicative Decrease) and continues with linear increase.

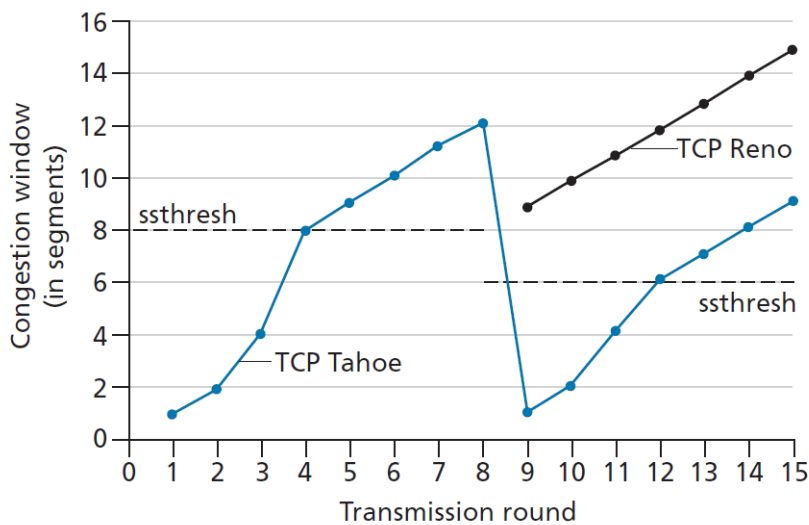


Figure 3: Comparison of cwnd Evolution: Tahoe vs. Reno

The AIMD Sawtooth

TCP Reno's behavior is known as ****Additive-Increase, Multiplicative-Decrease (AIMD)****. This results in the characteristic sawtooth pattern of bandwidth probing.

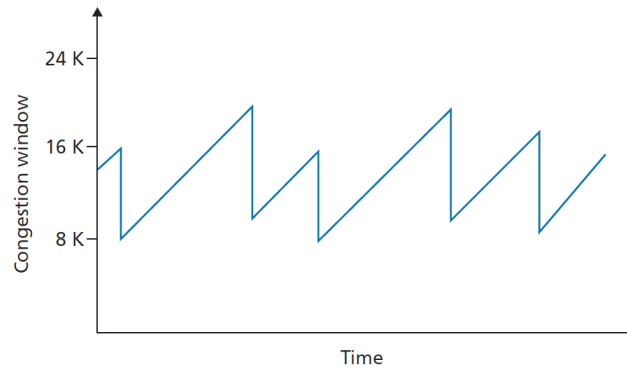


Figure 4: The AIMD "Sawtooth" Behavior

5. Optimized Probing: TCP CUBIC

TCP Reno can be overly cautious when ramping back up after a loss. **TCP CUBIC** (default in Linux) uses a cubic function for window growth based on the distance to the previous maximum window (W_{max}).

- **Rapid Ramp-up:** Quickly grows near W_{max} .
- **Cautious Probing:** Growth slows down significantly when nearing W_{max} to avoid early loss.
- **Aggressive Search:** If no loss occurs at W_{max} , it quickly explores higher rates.

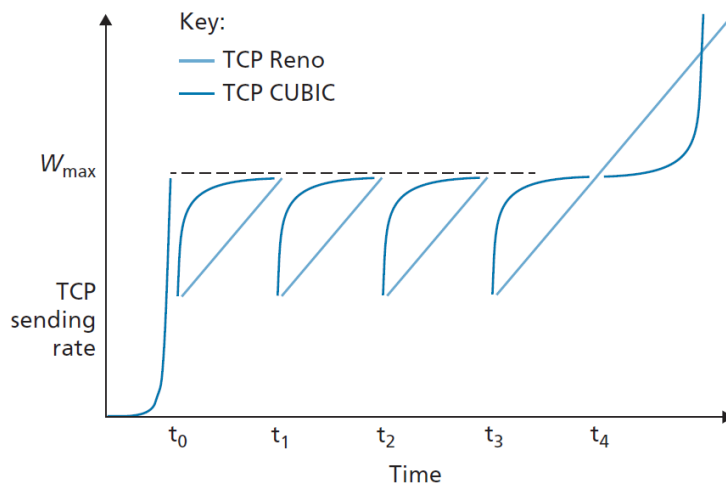


Figure 5: Congestion Avoidance: TCP Reno vs. TCP CUBIC

6. Steady-State Throughput Model

For long-lived Reno connections, ignoring the short slow-start periods, we can model the average throughput based on the maximum window size (W) and RTT.

Macroscopic Throughput Formula

$$\text{Average Throughput} = \frac{0.75 \cdot W}{\text{RTT}}$$

7. Summary

TCP Congestion Control Highlights

- **Self-Clocking:** Uses ACKs to trigger window updates.
- **Slow Start:** Quick start with exponential growth.
- **AIMD:** Additive increase, halves on 3-dup-ACK (Reno).
- **Fast Recovery:** Optimizes throughput by avoiding Tahoe's "restart from 1".
- **CUBIC:** Modern standard that optimizes the ramp-up phase.